WEEK 6 CYBERSECURITY INTERNSHIP REPORT

Web Security & Vulnerability Assessment

Submitted by:
Amaan Fayaz

Internship Program:
DG Interns Hub — Cybersecurity Internship

Week:
Week 6

Project Title:
Web Security and Vulnerability Testing (VAPT Basics)

Tools Used:

- Burp Suite Community Edition
- OWASP ZAP 2.17.0
- Damn Vulnerable Web Application (DVWA)
- Docker

Tasks Completed:

1. Study of OWASP Top 10 vulnerabilities
2. Installation and setup of web security testing tools
3. SQL Injection testing
4. Reflected and Stored Cross-Site Scripting (XSS) testing
5. HTTP request and response analysis using Burp Suite
6. Vulnerability scanning using OWASP ZAP
7. Documentation of findings and mitigation measures

Submitted to:
DG Interns Hub

Submission Date:
14th October 2026

Task 1: OWASP Top 10 — Selected Vulnerabilities

1. SQL Injection (SQLi)

SQL Injection is a web security vulnerability where an attacker inserts malicious SQL commands into an application's input fields. If the application does not properly validate or sanitise the input, the malicious SQL can be executed by the database.

How it works:
Web applications often use SQL queries to retrieve information from databases. If user input is directly included in a query, an attacker may manipulate that input to change the query's intended behaviour.

Possible impact:

- Unauthorized access to database information
- Bypass of authentication
- Modification or deletion of data

Prevention:

- Use prepared statements/parameterised queries
- Validate and sanitise input
- Apply least-privilege database permissions

2. Cross-Site Scripting (XSS)

Definition:
Cross-Site Scripting is a vulnerability in which malicious script content is injected into a trusted website and then executed in another user's browser.

Possible impact:

- Theft of session information
- Unauthorized actions on behalf of users
- Modification of webpage content

Prevention:

- Properly encode output
- Validate and sanitise user input
- Use appropriate Content Security Policy (CSP)

3. Broken Authentication

Definition:
Broken Authentication occurs when an application does not properly implement authentication or session management. This can allow attackers to compromise accounts or impersonate legitimate users.

Possible impact:

- Account takeover
- Unauthorized access
- Identity impersonation

Prevention:

- Enforce strong passwords
- Implement multi-factor authentication
- Use secure session management

4. Security Misconfiguration

Definition:
Security Misconfiguration occurs when a system, application, server, or security setting is incorrectly configured or left with insecure default settings.

Possible impact:

- Unauthorized access
- Information disclosure
- Increased attack surface

Prevention:

- Change default credentials
- Disable unnecessary services
- Keep software updated
- Use secure configuration standards

Task 2: Install & Setup Tools

Objective

The objective of this task was to install and configure web security testing tools for practicing Vulnerability Assessment and Penetration Testing (VAPT). Burp Suite Community Edition and OWASP ZAP were installed for web traffic analysis and vulnerability scanning. DVWA was configured as a local practice environment for testing common web vulnerabilities.

Tools Used

1. Burp Suite Community Edition

Burp Suite is a web application security testing tool used to intercept, inspect, and analyze HTTP requests and responses between a browser and a web application. It helps security testers understand how web applications handle user input and communicate with servers.

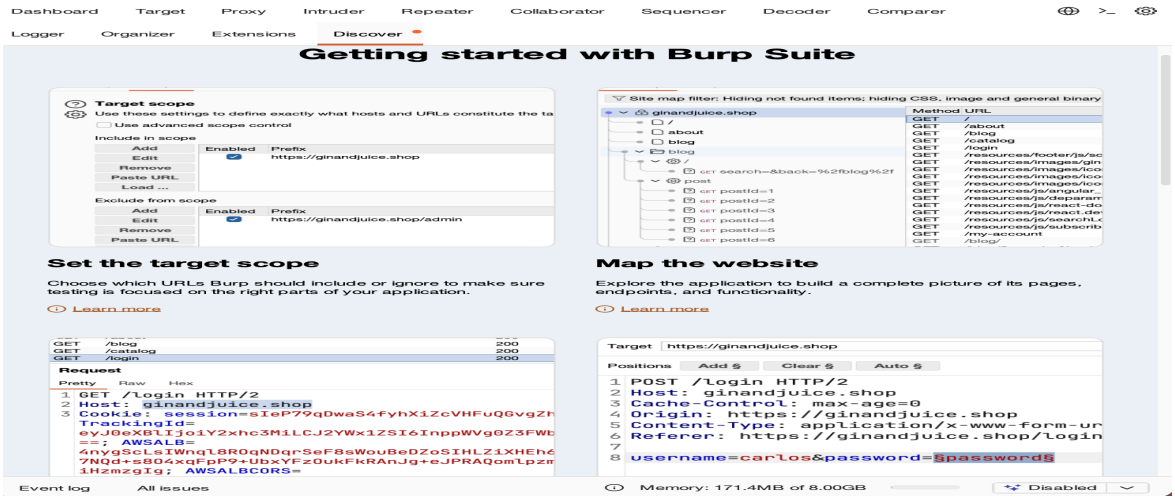


Figure 1: Burp Suite Community Edition Installation

OWASP ZAP

OWASP ZAP (Zed Attack Proxy) is an open-source web application security testing tool used to identify potential vulnerabilities. It supports automated scanning and manual testing, helping security testers examine web applications and understand common security issues.

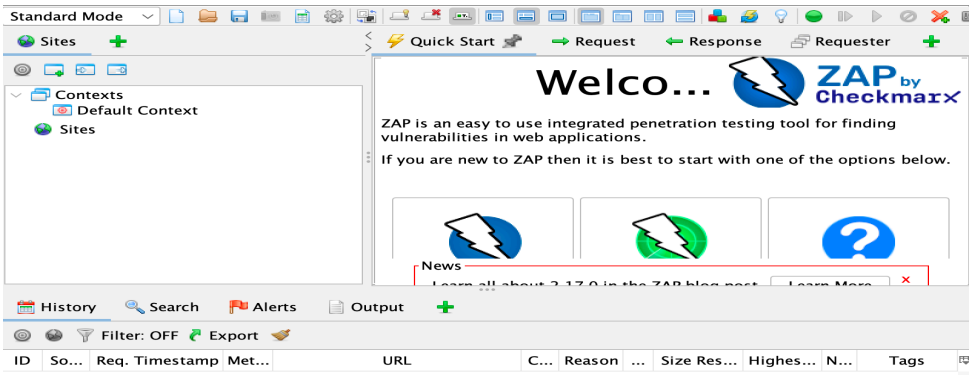


Figure 2: OWASP ZAP Installation

DVWA (Damn Vulnerable Web Application)

DVWA is a deliberately vulnerable web application designed for educational purposes. It provides a controlled environment for practising web application security testing and understanding common vulnerabilities.

Installation and Configuration:

- Downloaded the DVWA Docker image.
- Created and started a Docker container named dvwa.
- Configured local access through http://127.0.0.1:4280.
- Initialised the DVWA database.
- Set the security level to Low for beginner-level testing.

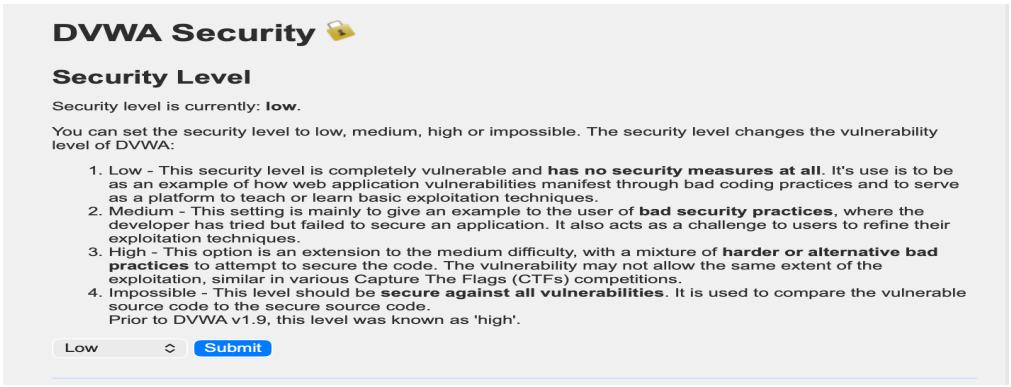


Figure 3: DVWA Security Level Configuration

The installation and configuration of Burp Suite Community Edition, OWASP ZAP, and DVWA were completed successfully. DVWA is running locally through Docker, and its security level is set to Low. The environment is ready for practicing SQL Injection and Cross-Site Scripting (XSS) and inspecting HTTP requests using Burp Suite.

Task 3: Test SQL Injection

Objective: The objective of this task was to identify and understand SQL Injection vulnerabilities in the Damn Vulnerable Web Application (DVWA). The test was performed in a controlled local environment with the security level set to Low.

Testing Procedure

The following steps were performed:

1. Opened DVWA in the browser using the local Docker address.
2. Navigated to the SQL Injection module.
3. Submitted a normal input (1) to observe the application's response.
4. Entered a SQL Injection test payload to examine how the application handled user input.
5. Observed the output and recorded the results using screenshots.

Payload Used: 1' OR '1'='1' #

This payload attempts to modify the SQL query's condition so that it returns multiple user records rather than a single matching record.

Results and Observations

The normal input returned the record for the user with ID 1. When the test payload was submitted, DVWA displayed multiple user records, including admin, Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith.

This demonstrated that the application was vulnerable to SQL Injection at the Low security level because the supplied input affected the database query.

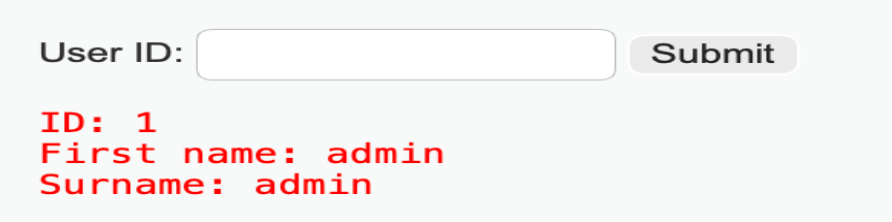


Figure 5: DVWA SQL Injection Module — Normal Input Test



Figure 6: DVWA SQL Injection Module — SQL Injection Payload Result

Risk and Impact of SQL Injection

SQL Injection is a serious web application vulnerability that occurs when user input is improperly handled in database queries. It may allow attackers to access sensitive information, bypass application controls, or modify database records, depending on the vulnerability and database permissions.

Mitigation Measures

The following security measures can help prevent SQL Injection:

1. Prepared Statements**:** Use parameterised queries to separate SQL commands from user input.
2. Input Validation: Validate all user input on the server side.
3. Least Privilege: Restrict database accounts to the permissions they require.
4. Secure Error Handling: Avoid exposing detailed database errors to users.
5. Regular Security Testing: Test applications regularly to identify and fix vulnerabilities.

Testing Conclusion

The SQL Injection test was successfully completed in the local DVWA environment. Normal input returned to one user's record, while the crafted payload returned multiple records. This demonstrated the SQL Injection vulnerability under the tested Low security configuration.

Cross-Site Scripting (XSS) Testing

Objective: To understand and test Reflected Cross-Site Scripting (XSS) and Stored Cross-Site Scripting vulnerabilities using DVWA in a controlled local environment.

Tools Used:

- Damn Vulnerable Web Application (DVWA)
- Web browser
- Burp Suite Community Edition

Reflected Cross-Site Scripting (XSS)

Objective: To check whether the application executes JavaScript entered through user input and immediately reflects it in the response.



Figure 7: DVWA Reflected XSS — Successful JavaScript Execution

Results and Observations

The Reflected XSS test was performed using the payload:

<script>alert('XSS Test')</script>

After submitting the payload, the browser displayed a pop-up alert containing “XSS Test”. This confirmed that the application executed the supplied JavaScript in the browser, demonstrating a Reflected Cross-Site Scripting vulnerability in the tested DVWA configuration.

Risk and Impact

Reflected XSS may allow an attacker to execute malicious JavaScript in a victim’s browser through specially crafted links or input. Depending on the application, this could lead to phishing, page manipulation, or exposure of information accessible to the victim.



Figure 8: DVWA Stored XSS — Successful JavaScript Execution

Results and Observations: The Stored XSS test was performed using the DVWA guestbook form. The JavaScript payload <script>alert('Stored XSS Test')</script> was entered into the Message field and submitted. The browser displayed a pop-up alert containing “Stored XSS Test”, confirming that the submitted script executed when the application processed the guestbook entry.

Risk and Impact: Stored XSS occurs when an application stores untrusted input and later executes it in a user’s browser. It may affect other users who view the affected content and could enable phishing, page manipulation, or exposure of information accessible to the browser.

Testing Conclusion: The Stored XSS test successfully demonstrated that JavaScript could be executed through content submitted to the DVWA guestbook. This highlights the importance of securely handling user input and encoding output before displaying it.

HTTP Request Analysis Using Burp Suite

Objective: To capture and analyze HTTP requests and responses generated during testing of the local DVWA application using Burp Suite Community Edition.

Tools Used:

- Burp Suite Community Edition
- Web browser
- Damn Vulnerable Web Application (DVWA)

Procedure

1. Open Burp Suite Community Edition.
2. Navigate to the Proxy tab.
3. Open HTTP history to view captured HTTP traffic.
4. Access the local DVWA application at http://127.0.0.1:4280 using a browser configured to work with Burp Suite.
5. Submit a test value through the DVWA SQL Injection form.
6. Locate the corresponding request in Burp Suite and inspect the request and response.

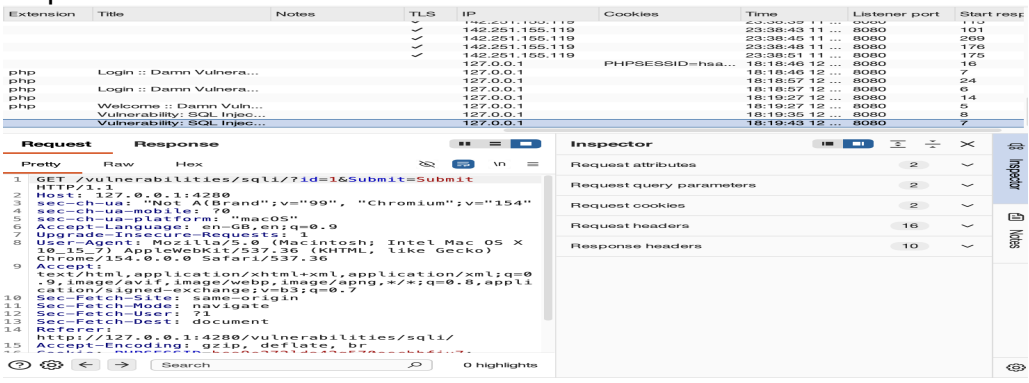


Figure 9: Burp Suite — Capturing a DVWA SQL Injection Request

Results and Observations

Burp Suite Community Edition successfully captured an HTTP GET request generated by the DVWA SQL Injection module. The request contains the URL path /vulnerabilities/sqli/ and the query parameters id=1 and Submit=Submit. The Host header identifies the local DVWA application at 127.0.0.1:4280.

This practical demonstrated how Burp Suite can intercept and inspect HTTP requests to understand how user input is transmitted to a web application.

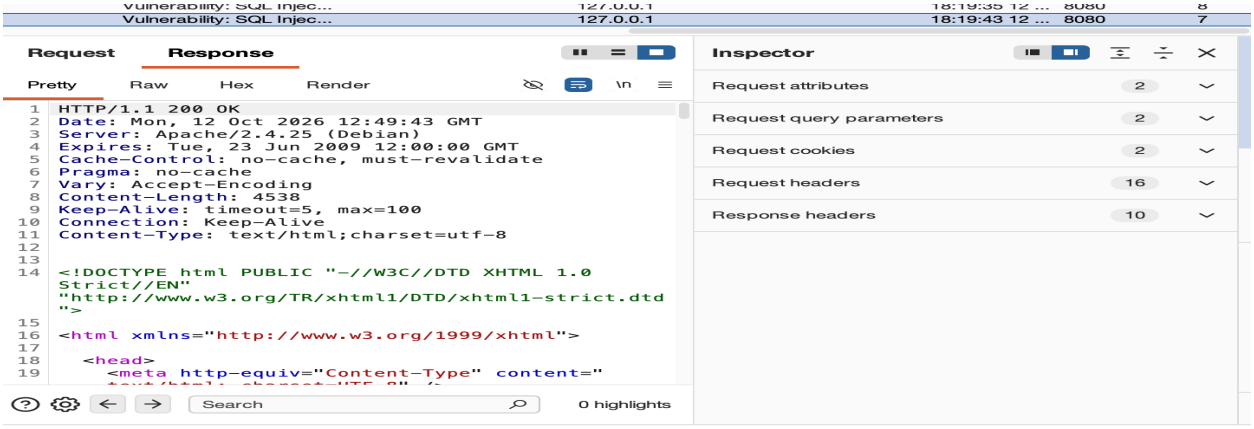


Figure 10: Burp Suite — DVWA HTTP Response Analysis

Response Analysis

The HTTP response was inspected in Burp Suite to observe the information returned by the DVWA application after the SQL Injection form was submitted. Examining the response helps testers understand application behaviour and identify potentially insecure handling of user input.

Task 4: OWASP ZAP Vulnerability Scanning

Vulnerability Scanning Using OWASP ZAP

Objective: To identify potential web application vulnerabilities by scanning the local DVWA application using OWASP ZAP.

Tools Used:

- OWASP ZAP 2.17.0
- Damn Vulnerable Web Application (DVWA)
- Web browser

Procedure:

1. Open OWASP ZAP.
2. Ensure DVWA is running locally at http://127.0.0.1:4280.
3. In ZAP, select the option to start an automated scan, if available.
4. Enter the local DVWA address as the target.
5. Start the scan and wait for it to finish.
6. Review the alerts and their risk levels.
7. Export the scan report.

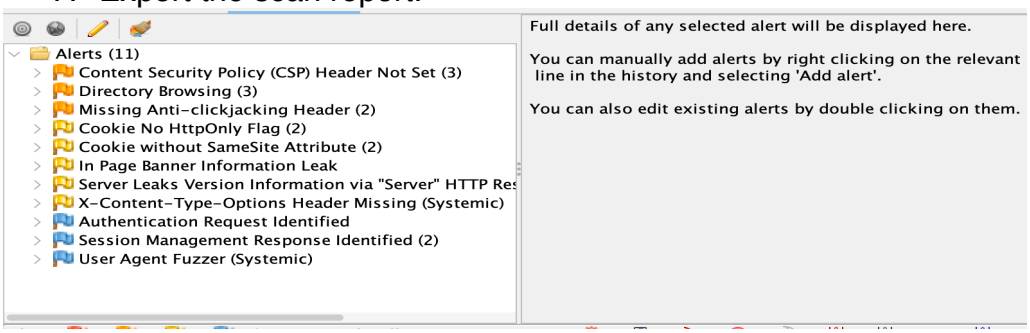


Figure 11: OWASP ZAP — Automated Vulnerability Scan Results

Results and Observations

OWASP ZAP was used to scan the local DVWA application at http://127.0.0.1:4280. The scan identified 11 alert categories related to security headers, directory browsing, cookie attributes, server information disclosure, and other application behaviours.

These alerts indicate areas that require further investigation and security improvement. Informational alerts provide useful context but do not necessarily represent exploitable vulnerabilities.

Key Findings:

1. Missing Security Headers: The scan reported missing Content Security Policy and anti-clickjacking headers.
2. Cookie Security: Cookie alerts indicated missing or insufficient security attributes.
3. Directory Browsing: The scanner reported a potential directory-listing issue that should be verified.
4. Information Disclosure: Server and page-banner information may reveal technical details about the application.

Finding 1: Content Security Policy (CSP) Header Not Set

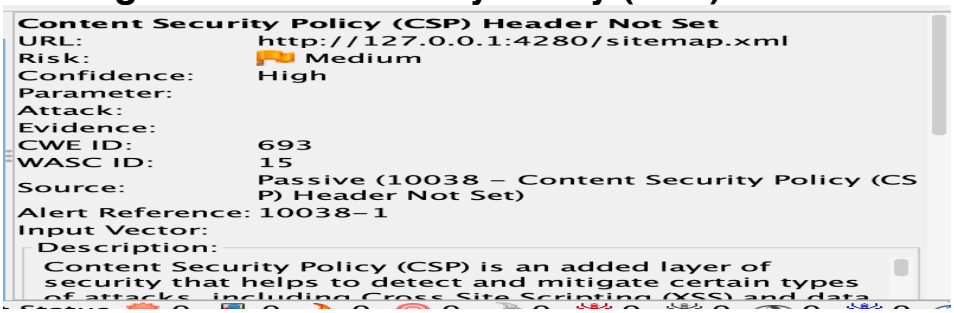


Figure 12: OWASP ZAP — Content Security Policy Header Not Set Alert

Description:
OWASP ZAP reported that the response from the local DVWA application did not include a Content Security Policy (CSP) header. CSP allows a web application to restrict the sources from which browsers can load scripts and other resources, helping reduce the impact of certain attacks, including some Cross-Site Scripting (XSS) attacks.

Potential Impact:
Without a suitable CSP, the browser may have fewer restrictions on loading and executing content. This can increase the impact of certain injection vulnerabilities if they are present.

Evidence:
The ZAP alert identified the missing CSP header in response from http://127.0.0.1:4280/sitemap.xml.

Mitigation Measures

1. Configure an appropriate Content Security Policy header on the web server or application.
2. Restrict scripts and other resources to trusted sources.
3. Avoid unsafe inline JavaScript where possible.
4. Test the policy to ensure that legitimate application functionality continues to work.

Finding 2: Missing Anti-Clickjacking Header

Description:
OWASP ZAP identified a missing anti-clickjacking protection header in the DVWA application response. This protection helps prevent attackers from embedding a website in another page to trick users into clicking elements without their knowledge.

Potential Impact:
If the application is vulnerable to clickjacking, attackers may trick users into interacting with application features unintentionally.

Mitigation Measures:

1. Configure the Content-Security-Policy header with an appropriate frame-ancestors directive.
2. Configure the X-Frame-Options header where appropriate.
3. Verify the security headers in the application's HTTP response.

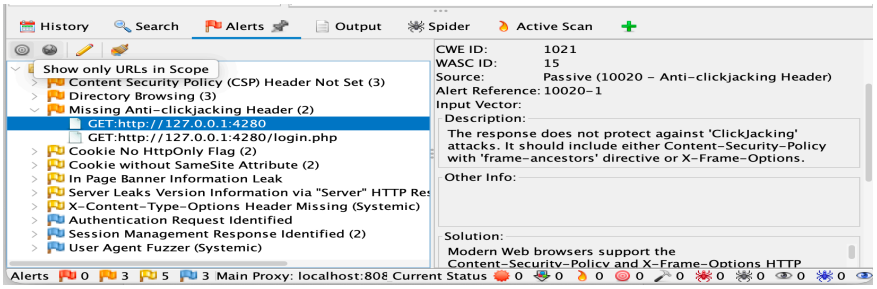


Figure 13: OWASP ZAP — Missing Anti-Clickjacking Header Alert

Evidence:
OWASP ZAP reported that the response from http://127.0.0.1:4280 did not include the X-Frame-Options header. The alert recommends using either this header or a suitable Content Security Policy frame-ancestors directive to help protect against clickjacking.

Conclusion

OWASP ZAP was used to scan the local DVWA application and identify potential security weaknesses. The scan identified issues involving security headers, cookie attributes, directory browsing, and information disclosure. The findings highlighted the importance of secure application configuration, appropriate security headers, and regular vulnerability assessments.

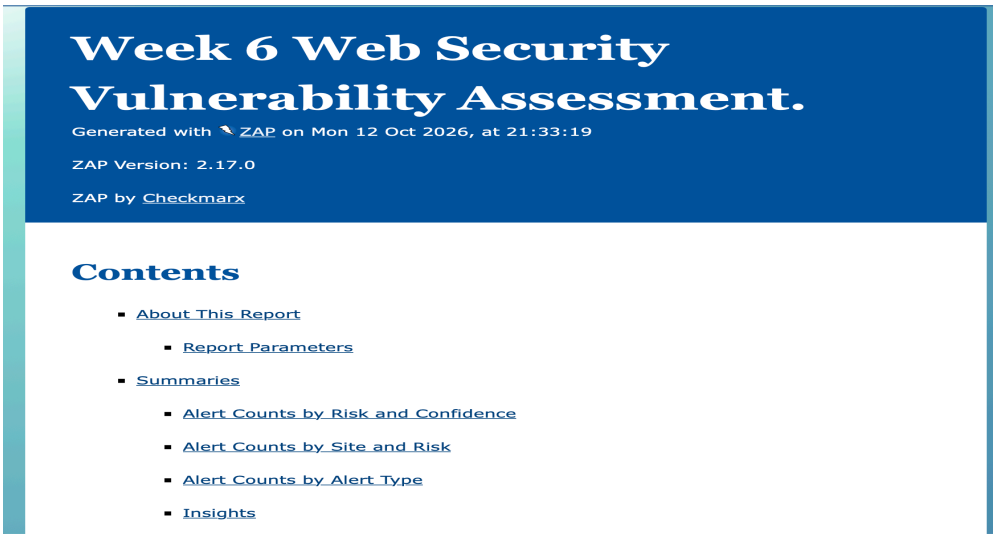
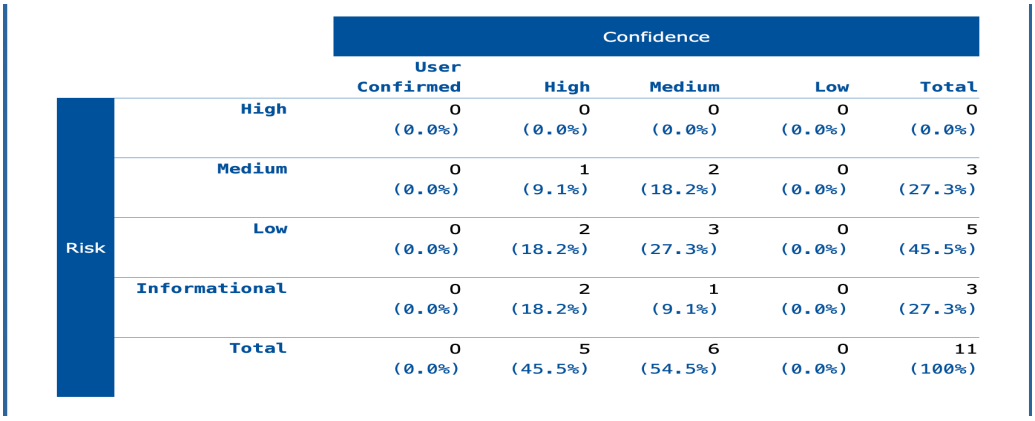


Figure 14: OWASP ZAP — Generated Vulnerability Assessment Report

Description:
The OWASP ZAP HTML report was generated successfully using version 2.17.0. It contains the scan summary, alert counts by risk and confidence, and details of the security issues identified during testing of the local DVWA application.



		Confidence				
		User Confirmed	High	Medium	Low	Total
Risk	High	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)
	Medium	0 (0.0%)	1 (9.1%)	2 (18.2%)	0 (0.0%)	3 (27.3%)
	Low	0 (0.0%)	2 (18.2%)	3 (27.3%)	0 (0.0%)	5 (45.5%)
	Informational	0 (0.0%)	2 (18.2%)	1 (9.1%)	0 (0.0%)	3 (27.3%)
	Total	0 (0.0%)	5 (45.5%)	6 (54.5%)	0 (0.0%)	11 (100%)

Figure 15: OWASP ZAP — Alert Counts by Risk and Confidence

Results and Observations:

The OWASP ZAP report identified 11 alerts during the scan of the local DVWA application. The results included 3 medium-risk alerts, 5 low-risk alerts, and 3 informational alerts. No high-risk alerts were reported.